

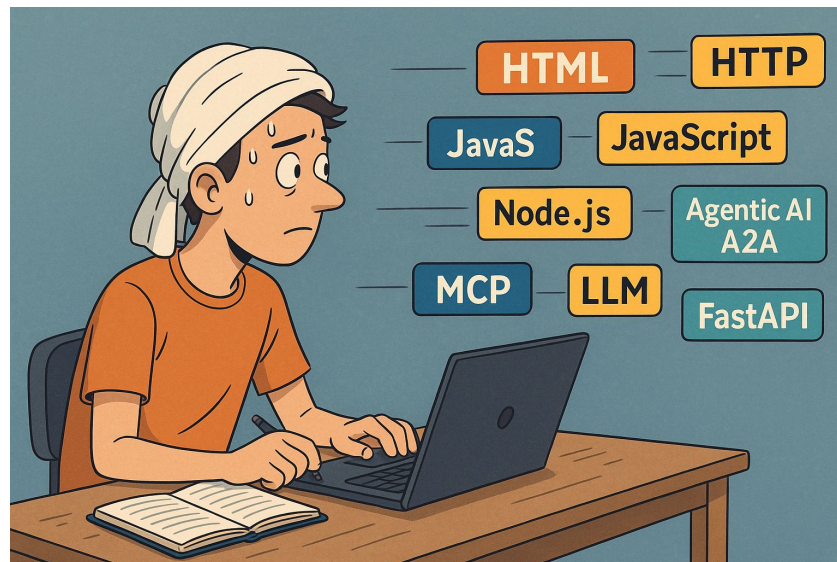
Software Development 2.0

Traditional software
development

Machine
learning

Coming 4 weeks are critical for success in the course

- You are learning HTML, HTTP, JavaScript, React, REST, FastAPI, Agentic AI A2A, MCP, LLM, etc
- make sure to code every day



JavaScript arrow functions: useful, but not a drop-in replacement

Where arrows shine

Use them for short callbacks that do not need their own **this**.

```
items.map(x => x.id)  
fetch(url).then(r => r.json())
```

Where regular functions are safer

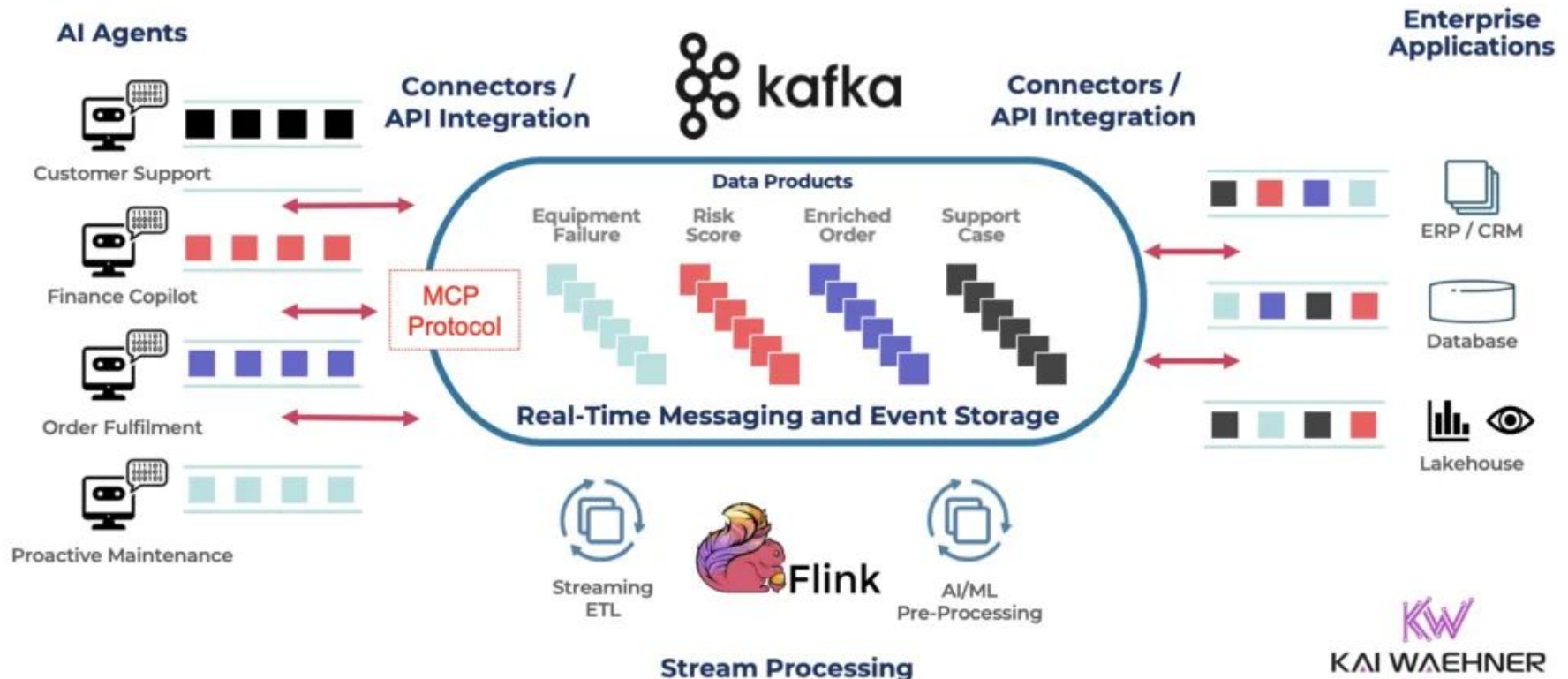
Use function when the caller, object, constructor, or framework must control context.

```
button.addEventListener('click', function () {  
  this.disabled = true;  
});
```

Rule of thumb: use arrow functions when you want a short function and you do not need your own this, arguments, or constructor behavior. Use traditional function syntax when those behaviors matter.

Agentic AI: Distributed Microservices with kafka

Agentic AI with Apache Kafka as Event Broker



Web APIs: HTTP → REST → FastAPI

Goal: design RESTful resources and implement them cleanly in FastAPI

We'll connect: methods → URIs → representations → status codes → validation → auth

HTTP: <https://en.wikipedia.org/wiki/HTTP>

HTTP stands for "Hypertext Transfer Protocol" and is the primary protocol used to transfer data across the web, essentially defining the rules for how web browsers and servers communicate with each other to display web pages and access online content.

- **Client-server model:**

HTTP operates on a client-server model where a client (like a web browser) sends a request to a server, which then responds with the requested data.

- **Stateless protocol:**

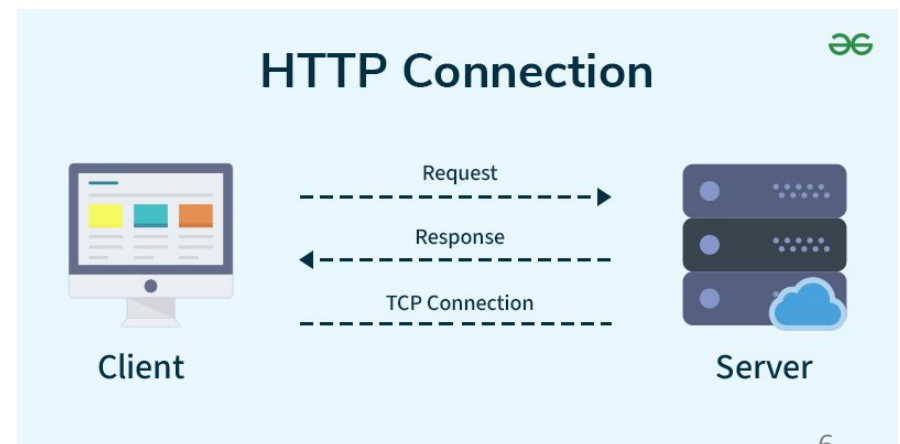
each request is independent, Messages have headers + optional body
Status codes communicate outcome (2xx/4xx/5xx)

- **Data transfer:**

HTTP is used to transfer various types of data like text, images, videos, and application data.

- **Request-response cycle:**

The basic interaction involves a client sending an HTTP request to a server, which then sends an HTTP response containing the requested data.



Rest Standards

Method	Meaning + key property
GET	Read a resource (safe, cacheable)
POST	Create/submit (not idempotent)
PUT	Create/replace at known URI (idempotent)
DELETE	Remove resource (idempotent-ish)

GET Method

- GET is used to request data from a specified resource.
- GET requests can be cached
- GET requests remain in the browser history
- GET requests can be bookmarked
- GET Avoid for sensitive data
- GET requests have length restrictions

Properties: safe  idempotent  cacheable 

- Common status codes: 200 OK, 304 Not Modified, 404 Not Found

POST Method

- The data sent to the server with POST is stored in the request body of the HTTP request.
- POST requests are not cached
- POST requests do not remain in the browser history
- POST requests cannot be bookmarked
- Properties: safe  idempotent  cacheable 
- Common status codes: 201 Created, 200 OK, 400 Bad Request, 409 Conflict

PUT Method

- PUT is used to send data to a server to create/update a resource.
- Create or replace a resource at a known URI.
- Calling the same PUT multiple times results in the same final state.
- Properties: safe  idempotent  cacheable 
- Common status codes: 200 OK, 204 No Content, 400 Bad Request, 404 Not Found

DELETE Method

- Delete the specified resource.
- Repeated deletes should not change the final state (if already deleted).
- Properties: safe  idempotent 
cacheable 
- Common status codes: 204 No Content, 404 Not Found

Responding to various HTTP methods

Resource	POST (Create)	GET (Read)	PUT (Update)	DELETE (Delete)
/tasks	Create a new task	Return all tasks	Replace all tasks with new tasks.	Delete all tasks
/tasks/123	Create a task inside another task.	Return a specific task	Update this task if exist or create a new one.	Delete this task.

HTTP - Response Codes

Server sends response code for every client request. This communicates to clients what happened to the their request (Pass, fail, error etc).

Response code for each type of request

Range (From client perspective)	Status code	Method	Comments.
1XX (100 - 199)	* - Informational	*	Request is accepted and is in progress
2xx (200 - 299) - Success	200 - OK	GET	Request is served.
	201 - Created	POST / PUT	Whenever a server create new resource on client's request
	202 - Accepted	*	Server accepted the requested but will be processed later.
	204 - No Content	PUT	Server performed the request but has no data to return.

HTTP - Response Codes

Range (From client perspective)	Status code	Method	Comments.
3XX - Redirection	301 - Permanently moved	GET	Server redirects client to the new URL of the same resource.
4xx (400 - 499) - Client Side Problem	400 - Bad Request.	*	Client has used either wrong URL or parameters, or provided insufficient data in the request.
	401 - Unauthorized.	*	Client is unauthorized to make this request. (May be client have passed wrong credentials)
	404 - Not Found	GET	The request resource cannot be found.
5xx (500 - 599) - Server Side Problem	500 - internal server error.	*	Server has encountered some exception while serving the request.

REST

What is REST?

- Representational State Transfer
- Style of Software Architecture for WWW
- Based on Client and Server
- Request and Response are built on transfer of “representation” or “resources”
- Uses HTTP protocol

Resources

- Are just Concepts
- Located by URIs
- URIs tell client that there's a concept somewhere
- Client then asks for specific representation of the concept from the representations the server makes available
- Resource is any entity that you may want to expose and that can be named. Typically, Each entity in ER-Diagram can be thought of as a resource.
- E.g Webpage is a representation of a resource
contd..

REST guidelines (principles)

2 URI's per resource.

1. Listing (collection) - /v1/tasks
2. Detail - /v1/tasks/{id}

Use nouns instead of verbs to name resources. So, instead of saying /v1/getAllTasks, we should say /v1/tasks.

This means, we need to revise all our URIs to follow rest principles.

Prefer plurals over singular: /tasks over /task.

For associations: /resource/{id}/resource In our case, we may have /user/8908/tasks, to get all the tasks for a user.

RESTful Web Services

- Representational State Transfer (REST)
 - Roy Fielding, 2000 (doctoral dissertation)
 - Examination of the Internet as a stateless service of near-limitless expansion model with a simple but effective information delivery system
- Key concepts
 - Resources - source of information
 - Consistent access to all resources
 - As in interface and communication – Not content or function
 - Stateless protocol
 - Hypermedia – links in the information to other data (connectedness)

Example

- Consider a Book store which has enabled a Web Service for book ordering.
- It should enable customers to
 - Get a list of books
 - Get detailed information on each book
 - Submit a order to purchase it

contd..

Get books list

- Client uses following URL to get books list

<http://www.bookstore.com/books>

- The server returns

```
<?xml version="1.0"?>
<b:Books xmlns:p="http://www.Books-store.com"
xmlns:xlink="http://www.w3.org/1999/xlink">
  <Book id="00345" xlink:href="http://www.Books-store.com/Books/00345"/>
<Book id="00346" xlink:href="http://www.Books-store.com/Books/00346"/>
<Book id="00347" xlink:href="http://www.Books-store.com/Books/00347"/>
<Book id="00348" xlink:href="http://www.Books-store.com/Books/00348"/>
</p:Books>
```

contd..

Get detailed information on each book

- The client then uses one of the book id and traverses to that resource.
- The client uses

<http://www.bookstore.com/books/00346>

The Server responds with

```
<?xml version="1.0"?>
<p:Book xmlns:b="http://www.Books-store.com"
xmlns:xlink="http://www.w3.org/1999/xlink">
  <Book-ID>00345</Book-ID>
  <Name>Harry Potter</Name>
  <Description>This boos is written by JK Rowling</Description>
  <Specification xlink:href="http://www.Books-store.com/books/00346/readings"/>
    <UnitCost currency="USD">20.0</UnitCost>
    <Quantity>10</Quantity>
</b:Book>
```

contd..

Creating a RESTful service

Steps

1. Define the domain and data
2. Organize the data in to groups
3. Create URI to resource mapping
4. Define the representations to the client
(XML, HTML, CSS, ...)
5. Link data across resources (connectedness or hypermedia)
6. Create use cases to map events/usage
7. Plan for things going wrong

Top



Down

When is it appropriate?

- The web services are completely stateless
- Caching infrastructure can be leveraged for performance
- The service producer and consumer have mutual understanding of context and content being passed e.g NetFlix Api
- Bandwidth is a constraint

What else can we expect from proper REST APIs?

Pagination:

Return subset of all the data present in DB. User must be able to tell you get me next 20 records after 60.

This can be achieved by adding offset and limit parameters to our GET APIs.
ex. /v1/tasks?offset=60&limit=20

Multiple Representation Formats:

Server must return the data in more than one format (XML, JSON etc).

For client to specify what format the server should reply, ACCEPT http request header needs to be set. Ex. ACCEPT : Application/xml.

For server to specify what format is being used in the response format, Content-Type http response header needs to be set.
Ex. Content-Type : Application/xml.

What else can we expect from proper REST APIs?

Filtering based on parameters:

Filtering acts as a WHERE clause in SQL statement. To get all tasks whose completion date is 1st May 2013.

ex. `/v1/tasks?completion-date=2013-05-01`

Ordering:

This plays a role of OrderBy clause in SQL statement. To get all the tasks sorted by priority.

ex. `/v1/tasks?order_by=priority` - Sorts in ascending order of priority.

`/v1/tasks?order_by=-priority` - Sorts in descending order of priority.

Web Application Architecture {MVC}

Presentation Layer

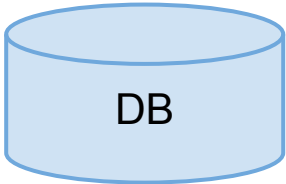
Takes care of Content Representation. (HTML5, CSS3).

Business Layer

Computational Logic on data using language libraries.

Data Access Layer
(SQL or ORM)

Access data from DB using RAW SQL queries or an ORM



DB

Data Storage System (SQLite, MySQL, ORACLE).

References

- Roy Fielding, 2000 (doctoral dissertation)
 - <http://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- <http://java.sun.com/developer/technicalArticles/WebServices/restful/>
- http://en.wikipedia.org/wiki/Representational_State_Transfer
- <http://www.devx.com/DevX/Article/8155>
- http://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm
- Rest Api <https://restfulapi.net/resource-naming/>

From REST design → FastAPI
implementation

What is FastAPI?

FastAPI is a modern, fast (high-performance), web framework for building APIs with Python 3.7+ based on standard Python type hints.

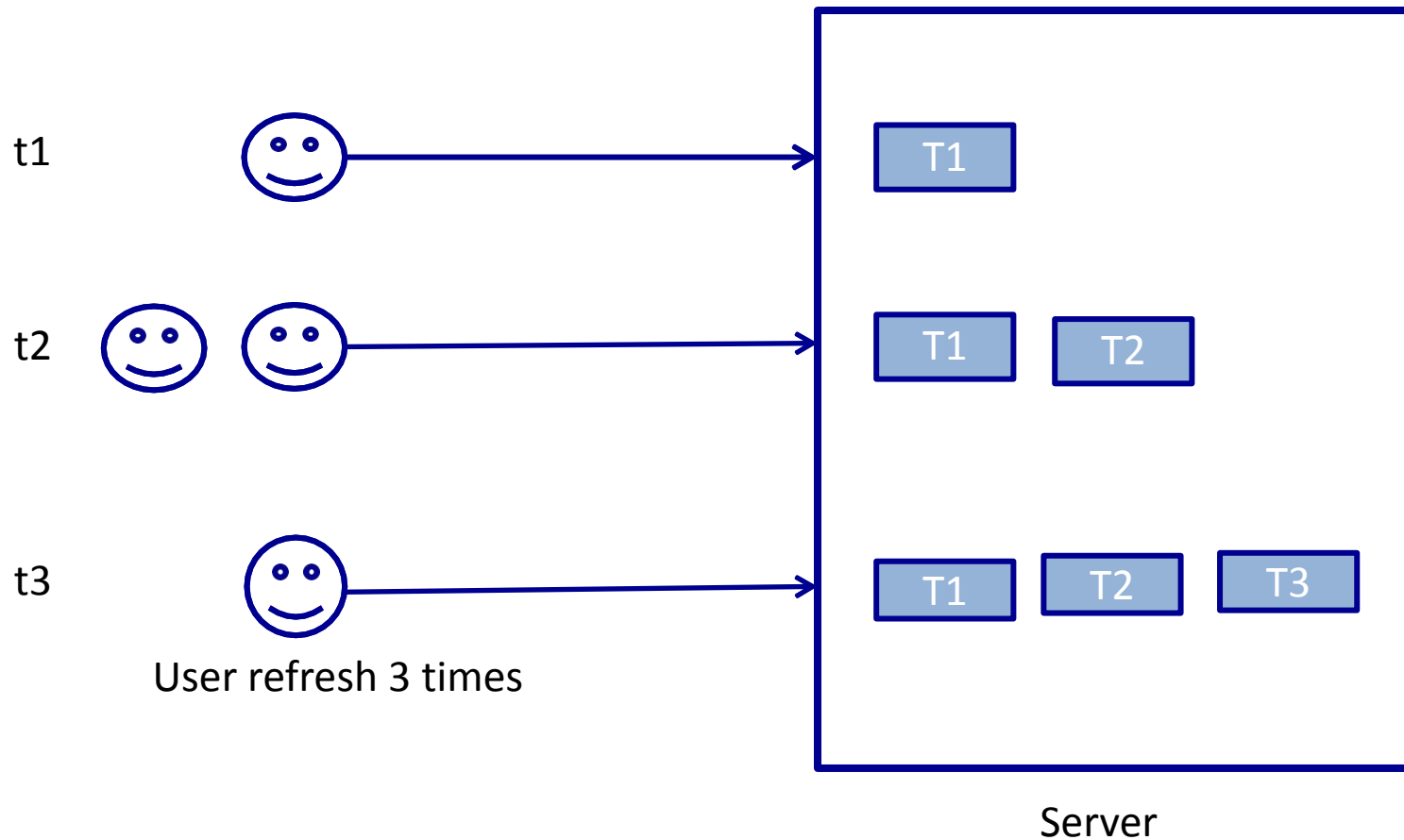
- A modern web framework for building APIs
- Based on Python type hints and Pydantic
- Supports asynchronous programming

Evolution of web

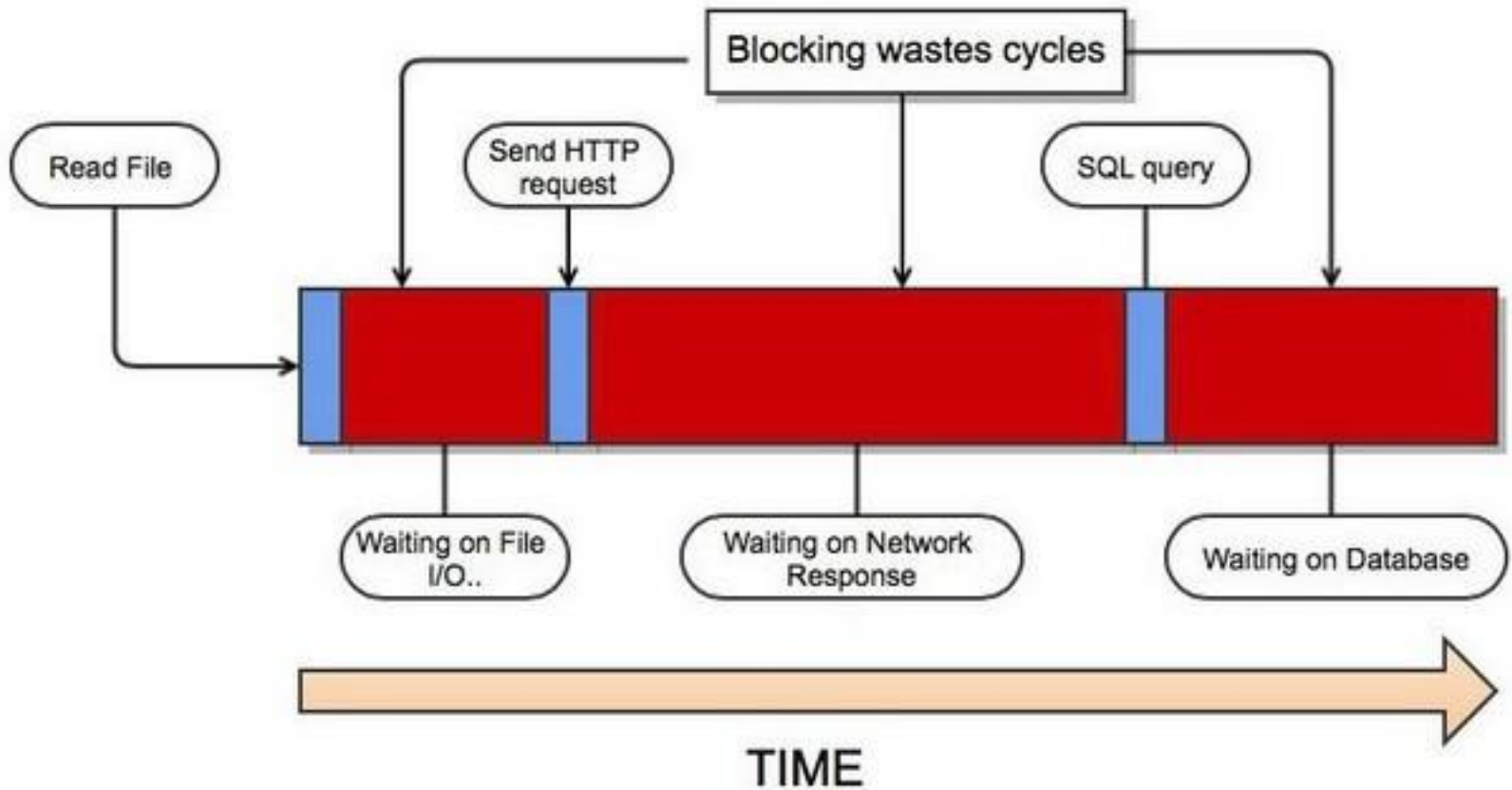
Web has evolved from

- Static websites (90's)
- Dynamic web applications (AJAX) (early 2000's)
- Real time web applications (notifications)

Traditional multi threaded server



Traditional (Blocking) Thread Model



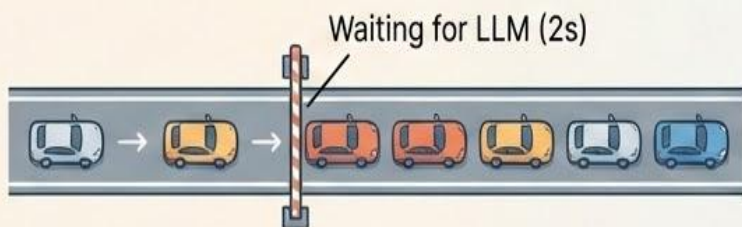
Why blocking hurts (I/O wait)

Most web time is waiting on I/O (DB, network, disk).
Blocking ties up a worker while waiting → lower throughput.
Goal: keep workers busy doing useful work.

FastAPI's Defining Feature: Asynchrony

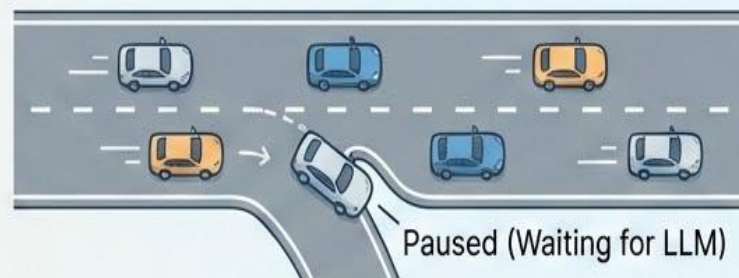
Non-blocking approach for handling thousands of concurrent connections.

Traditional Synchronous (Blocking)



Server “waits” for external tasks.
Cannot help others until done.

Modern Asynchronous (Non-Blocking)



Server moves to next request while waiting.
Context-switches back when ready.

The Core Concept: `async` and `await`

- Built on Starlette, a light ASGI framework.
- Prefix endpoint functions with the ``async`` keyword.

FastAPI: Asynchronous Agent Update Example

```
import asyncio
import httpx
from fastapi import FastAPI

app = FastAPI()

# A simulated external API call
async def fetch_data(service_name: str, delay: int):
    print(f"Requesting {service_name}...")
    await asyncio.sleep(delay) # Non-blocking wait
    return {"service": service_name, "status": "Ready"}

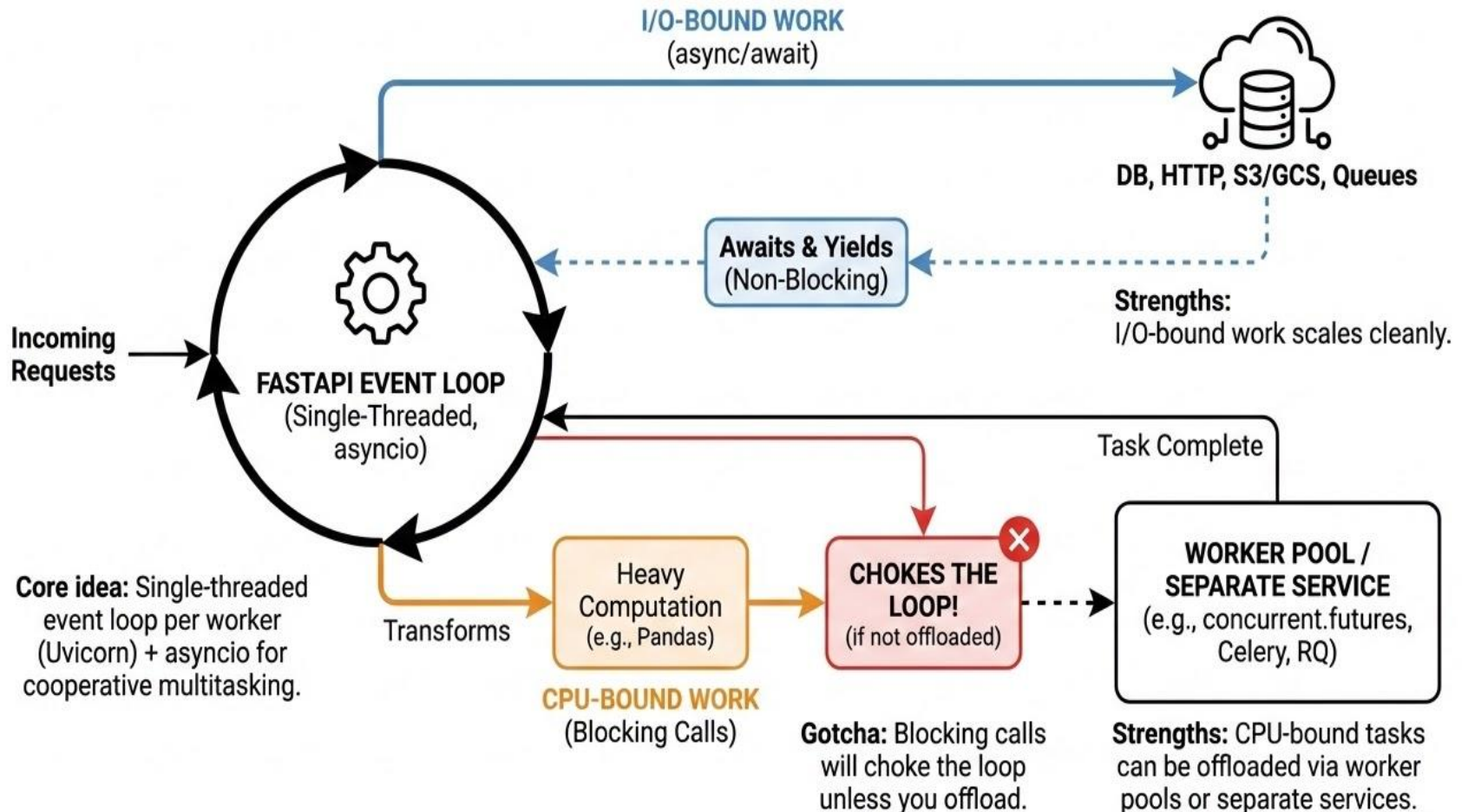
@app.get("/agent-update")
async def get_agent_update():
    # We trigger two tasks simultaneously
    # The server doesn't wait for 'weather' to finish before starting 'flights'
    weather_task = fetch_data("WeatherAPI", 3)
    flight_task = fetch_data("FlightAPI", 2)

    # 'gather' runs them concurrently and waits for the results
    results = await asyncio.gather(weather_task, flight_task)

    return {
        "message": "Agent has gathered all data",
        "data": results
    }
```

Event Loop Example

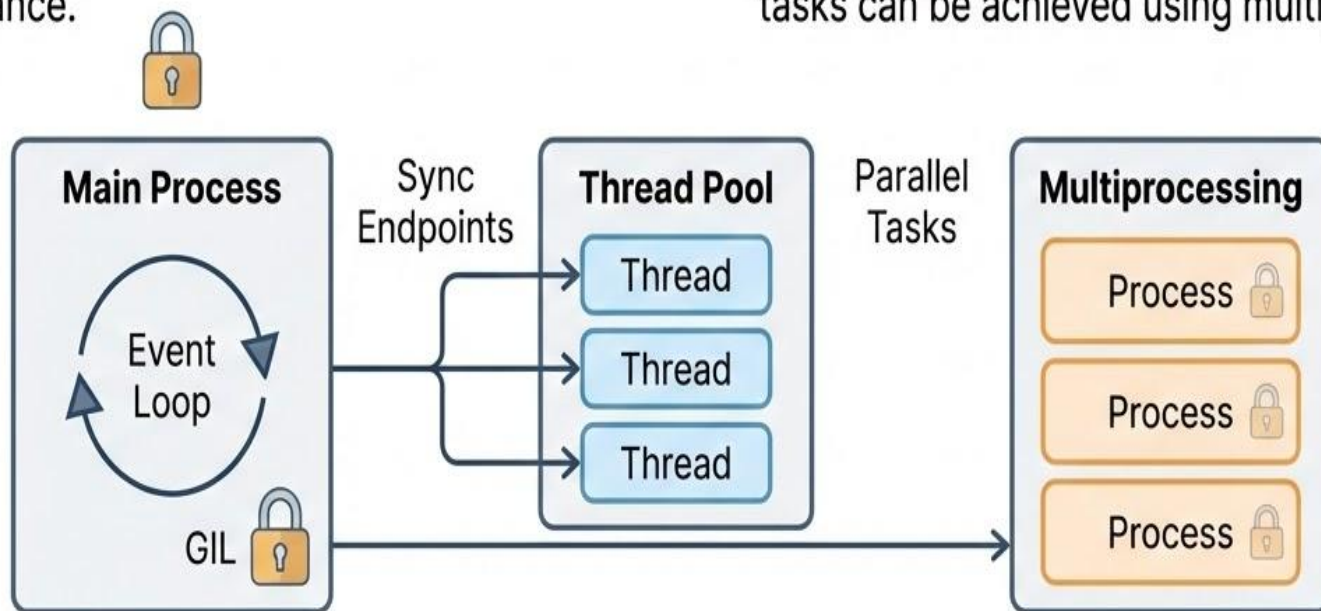
FastAPI: Asyncio and Cooperative Multitasking



FastAPI (Python): Handling CPU-Bound Tasks & The GIL

The Global Interpreter Lock (GIL)

- Python's GIL ensures only one thread executes Python bytecode at a time per process.
- This can cause CPU-bound tasks to block the single-threaded event loop, affecting performance.



FastAPI Solutions

- **Synchronous** def endpoints are automatically run in a separate thread pool to avoid blocking the loop.
- True **parallel processing** for CPU-intensive tasks can be achieved using multiprocessing.

The Code Snippet (main.py)

Python

```
from fastapi import FastAPI

# Create the application instance
app = FastAPI()

# Define a route for the root URL using the GET method
@app.get("/")
def read_root():
    return {"message": "Hello, FastAPI!"}
```

How to Run It

In modern FastAPI (version 0.111.0+), you don't necessarily need to type the full uvicorn command. You can use the new FastAPI CLI:

Installation: `pip install "fastapi[standard]"`

Run Development Server: ```bash
nfastapi dev main.py\n```

* Note: ``fastapi dev`` automatically enables auto-reload and sets the port to 8000.*

Key Points Explained

- **FastAPI() Instance:** This object is the main point of interaction to create your API. It handles all the routing, middleware, and settings.
- **Path Operation Decorator:** `@app.get("/")` tells FastAPI that the function below is responsible for handling requests that go to the path `/` using the GET HTTP method.
- **Automatic Serialization:** Notice that the function returns a Python dict. FastAPI automatically converts this into a JSON response for the browser or client.
- **The CLI (`fastapi dev`):** This command manages the Uvicorn server for you. It provides a beautiful terminal output that highlights your routes and any errors during development.

Next Level: Path Parameters

To make the API interactive, you can capture variables directly from the URL:

Python

```
@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id,
            "message": "Fetching data..."}
```

If you visit `/items/42`, FastAPI will verify that 42 is an integer. If you try `/items/abc`, it will automatically return a 422 Unprocessable Entity error, saving you from writing manual validation code.

FastAPI Path Parameters: Core Concepts & Best Practices

Type hints (e.g., `: int`) unlock powerful, automatic features in FastAPI.

Major Features from Type Hints

-  **Data Conversion:** Automatic string-to-type conversion (e.g., `/items/5` → `5` as an `int`).
-  **Data Validation:** Automatic error handling (e.g., `/items/foo` → 422 error).
-  **Documentation:** Auto-generated Swagger UI shows required types and constraints.

Advanced Path Validation

Use the `Path` class for more granular control (e.g., positive integers).

```
from fastapi import FastAPI, Path
app = FastAPI()
@app.get("/items/{item_id}")
async def read_item(
    item_id: int = Path(title="The ID", gt=0, le=1000)
):
    # gt=0: > 0, le=1000: <= 1000
    return {"item_id": item_id}
```

Path vs. Query Parameters

Type	Syntax	Usage
Path Parameter	<code>/items/{item_id}</code>	Identify a specific resource (Required).
Query Parameter	<code>/items/?skip=0</code>	Filter, sort, or paginate (Usually optional).

Order Matters

FastAPI matches routes in order. Define fixed paths *before* dynamic paths.

```
@app.get("/items/me") # Fixed path first
async def read_user_me():
    return {"user_id": "current_user"}

@app.get("/items/{item_id}") # Dynamic path second
async def read_item(item_id: str):
    return {"item_id": item_id}
```

FastAPI Request Body & Pydantic: The Powerful Contract

Pydantic creates a 'Contract' between client and server, defining data structure and validation rules.

How Pydantic Works its Magic

-  **Reading:** FastAPI reads the HTTP body as JSON.
-  **Validation:** Checks if JSON matches the Pydantic model (e.g., `name` is a string, `price` is a number).
-  **Conversion:** Converts JSON to a Python object, allowing dot notation access (e.g., `item.name`).
-  **Error Handling:** Automatically sends a detailed error message for invalid data (e.g., price as "expensive").

Extending the Model

```
from typing import Optional
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    description: Optional[str] = None # Optional field
    price: float
    tax: float = 0.0                 # Default value
    tags: list[str] = []             # List of strings
```

The "Why" for Agentic Behavior & A2A

-  **Reliability:** Pydantic acts as a gatekeeper, rejecting malformed or "hallucinated" JSON from AI Agents immediately.
-  **Self-Correction:** Detailed error schemas allow intelligent Agents to parse the error and retry with the correct format.

FastAPI + Pydantic = Auto Documentation

- Shows exactly what the JSON body should look like.
- Provides a "Try it out" button to generate JSON.
- Documents specific data types for every field in the Swagger UI (`/docs`).

FastAPI Dependency Injection (DI)

What it is: A way to declare what an endpoint *needs* (auth, DB session, config, shared params) and have FastAPI **automatically provide it** per request.

- **How it works:** Use `Depends()` to “inject” the return value of a dependency function into your path operation.
- **Why it matters:**
 - **Reusable** logic (auth, DB, validation) across routes
 - **Cleaner endpoints** (separate concerns)
 - **Testable** (override dependencies in tests)
 - Supports **dependency graphs** (dependencies can depend on other dependencies)
- **Cleanup support:** Dependencies can use `yield` for **setup + teardown** (e.g., open/close DB session).
- **Two common patterns:**
 - Inject as a **parameter**: `user = Depends(get_current_user)`
 - Run as a **route requirement**:
`dependencies=[Depends(require_admin)]`

FastAPI Dependency Injection (DI): Modularity & Testability

DI injects common logic (DB sessions, security, config) directly into endpoint functions, making FastAPI exceptional.

How it Works: The "Depends" Marker

-  **Requirement:** You tell FastAPI, "This function needs a token."
-  **Resolution:** FastAPI runs your `get_token` function first.
-  **Injection:** FastAPI passes the result to your route.

Shared Logic vs. Security

As a Function Parameter: Use when the return value is needed (e.g., current user).

```
@app.get("/items/")
def read_items(token: str = Depends(get_token)):
    return {"token_received": token}
```

In the Decorator: Use for "Gatekeepers" (logic only, no return value needed).

```
@app.get("/secure/",
dependencies=[Depends(get_token)])
def secure_route():
    return {"message": "You passed the check!"}
```

Why this is Critical for Agents

-  **Tool Authorization:** Verify agent permissions before high-cost actions.
-  **Database Cleanup:** Use `yield` for automatic resource management (open/close connection).
-  **Testing (Overrides):** Easily swap real dependencies with fakes using `app.dependency_overrides`.

Securing an agent-callable API

Your endpoint is a tool an agent can call — treat every request as untrusted.

- Auth is the canonical dependency: declare it once, apply it to any route.
- API key — simplest. Client sends a header; a dependency verifies it.

```
def require_key(x_api_key: str = Header(...)):
    if x_api_key not in ALLOWED:
        raise HTTPException(status_code=401, detail="Invalid key")
    return x_api_key

@app.post("/summarize", dependencies=[Depends(require_key)])
```
- OAuth2 bearer — for user identity. `OAuth2PasswordBearer(tokenUrl="token")`,
 - then `Depends(oauth2_scheme)` gives you the token to decode and verify.

Scope the agent, not just the user

- Give each agent its own credential with least privilege — a read-only
 - research agent should not be able to POST.
- 401 vs 403: 401 = we do not know who you are. 403 = we do, and you may not.
- Never pass secrets in the URL — they land in logs, history and referrers
 - (see the GET guidance earlier). Put them in headers.

FastAPI Query Parameters: Filtering & Pagination

Query Parameters are key-value pairs after the `?` in the URL, used for filtering, sorting, and pagination, distinct from Path Parameters.

How it Works & Combining Parameters

-  **Automatic Detection:** Not in `{}`, so FastAPI assumes query parameter.
-  **Default Values:** Setting a default makes it optional.
-  **Type Hinting:** Automatic conversion and validation (string to type).

When to Use Query Parameters

- **Filter results:** (e.g., `?color=red`).
- **Paginate:** (e.g., `?page=2&size=50`).
- **Sort data:** (e.g., `?sort=price_desc`).
- **Toggle features:** (e.g., `?include_metadata=true`).

Combining Path and Query Parameters

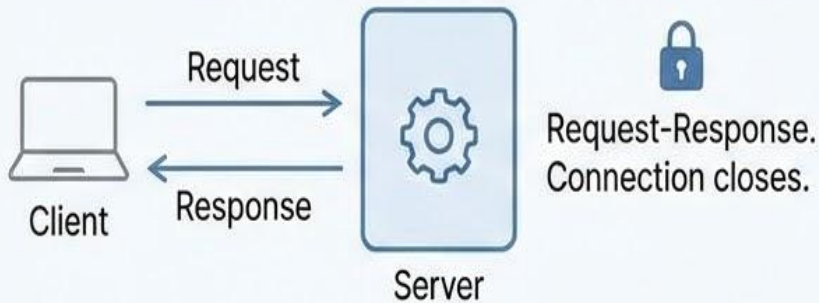
```
from fastapi import FastAPI
app = FastAPI()
@app.get("/items/{item_id}")
def read_item(item_id: int, q: str | None = None, short: bool = False):
    item = {"item_id": item_id}
    if q: item.update({"q": q})
    if not short: item.update({"description": "This is a long description..."})
    return item
```

- `item_id`: Path Parameter (Required).
- `q`: Query Parameter (Optional string).
- `short`: Query Parameter (Boolean, smart interpretation: `1`, `True`, `true`, `on`).

WSGI vs. ASGI: The Evolution of Python Web Servers

1. WSGI (Web Server Gateway Interface) 🕒

Created in 2003, standard for the “Synchronous” era.



The Limitation: Handles one request per thread.

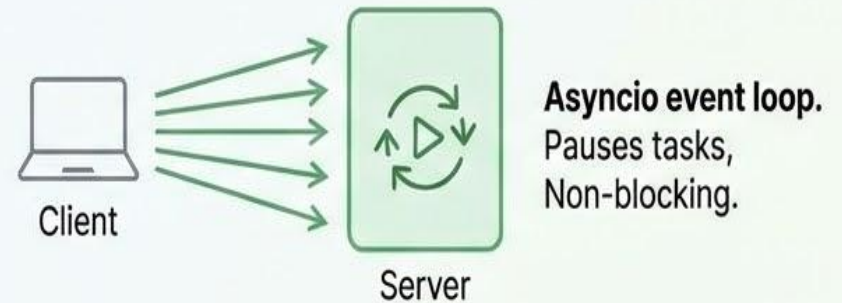
🚫 Blocks while waiting (e.g., DB, LLM).

Best for: Standard web apps like Django (traditional) or

Best for: Standard web apps like Django (traditional) or Flask.

2. ASGI (Asynchronous Server Gateway Interface) ⚡

Spiritual successor, built for **concurrency** on a single thread.



The Superpower: Long-lived connections. Essential for:

💬 **WebSockets:** Real-time chat or live data feeds.

⚙️ **HTTP/2:** Faster, multiplexed communication.

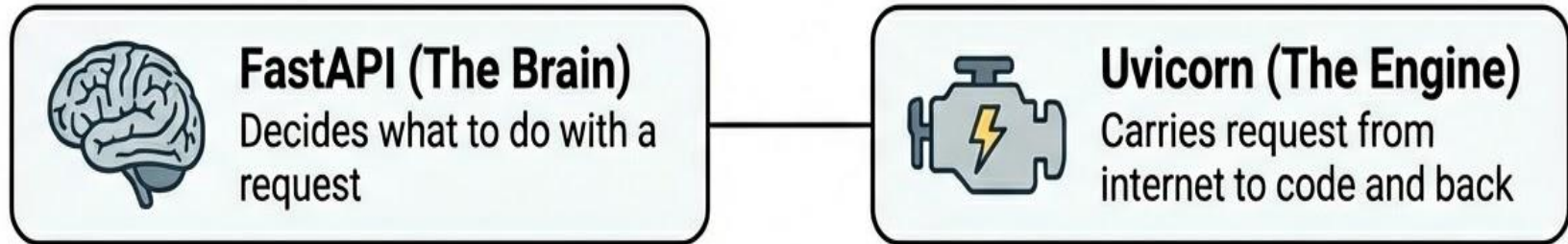
🗣️ **A2A Interactions:** Streaming agent thoughts, feedback.

Best for: FastAPI, Starlette, and modern Django (Channels).

Evolution

FastAPI is the web framework. Uvicorn is the ASGI server that runs it.
It is ASGI server implementation

Uvicorn & FastAPI: The Core Relationship



Why It's Needed



Speed

Built on uvloop and httptools. One of the fastest Python server implementations.



WebSockets & HTTP/2

Essential for real-time Agentic interactions.
Supports multiple simultaneous connections.



Low Latency

Crucial for A2A (Agent-to-Agent) communication.
Minimizes data transfer overhead.

Execution & Production Model

How It Works in Practice



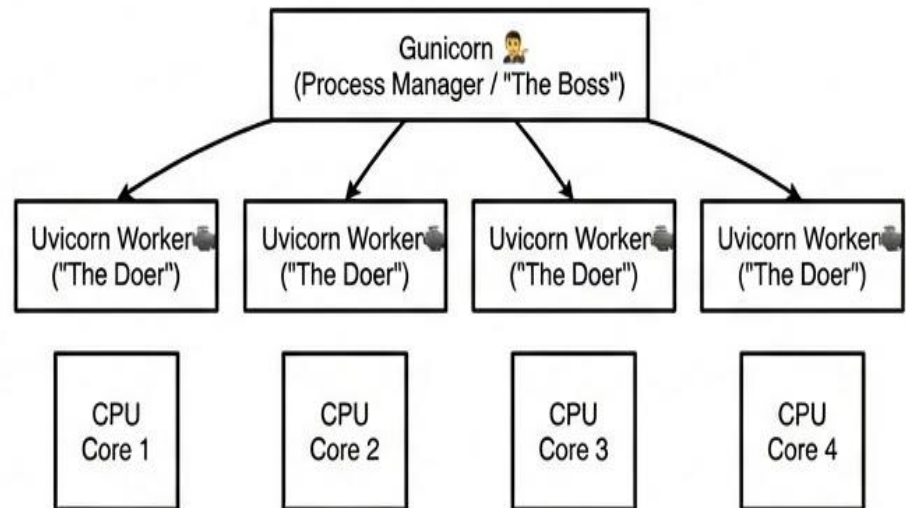
main: The name of your Python file (e.g., main.py).

app: The FastAPI object inside that file.

--reload: A development feature that restarts the server on code changes.

This command starts the Uvicorn server and runs your FastAPI application.

The 'Worker' Model in Production



Gunicorn manages multiple Uvicorn workers, distributing tasks across CPU cores to prevent heavy tasks from slowing down the entire API.

Running Your First FastAPI App

To get running, you need the Python code (**FastAPI**) and the server command (**Uvicorn**).

1. The Code (main.py)

Create `main.py` with two routes: a basic home page and a dynamic path parameter route.

```
from fastapi import FastAPI

# Initialize the app
app = FastAPI()

# 1. Basic GET route
@app.get("/")
def home():
    return {"status": "Online", "message": "Hello from FastAPI!"}

# 2. Dynamic GET route
@app.get("/greet/{name}")
def greet_user(name: str):
    return {
        "greeting": f"Hello, {name}!",
        "integration": "FastAPI + Uvicorn works!"
    }
```

2. Running the Server

Open your terminal in the file's folder and run:

```
uvicorn main:app --reload
```

- **main:** Refers to the file `main.py`.
- **app:** Refers to the variable `app = FastAPI()` inside that file.
- **--reload:** Restarts the server automatically on code changes.

3. Accessing the Web Response

Once started, you will see a message saying Uvicorn is running.

<http://127.0.0.1:8000>

Open this URL in your browser to see the JSON response from the home route.

References for learning FastAPI

- [FastAPI Tutorial \(User Guide\)](#) — step-by-step, covers most features and patterns.
- [FastAPI Security Tutorial \(OAuth2\)](#) — password flow, bearer tokens, common auth building blocks.
- [FastAPI Community Hub](#) — entry page pointing to docs/resources/ecosystem.
- [FastAPI GitHub repo](#) — examples, discussions, release notes.

Starlette (important for cookies/middleware/sessions in FastAPI)

- [Starlette Middleware docs](#) — how middleware works under the hood (FastAPI uses this).
- [Starlette Intro / dependencies](#) — includes note that `SessionMiddleware` relies on `itsdangerous`.

Practical tutorials (hands-on examples)

- [Real Python: FastAPI example app](#) — great walkthrough with real API patterns and testing mindset.
- [FastAPI Interactive \(hands-on, browser-based\)](#) — interactive lessons you can run while learning.

Server-side sessions (if you want Redis-backed sessions)

- [starlette-session \(PyPI\)](#) — a library providing server-side session backends (e.g., Redis).

Agentic Behavior & Agent-to-Agent (A2A) Interactions

What "agentic" really means

Simple idea: a loop of

Plan → Do → Check → Adjust/Act (think PDCA).

It's a continuous improvement cycle first developed by W. Edwards Deming, widely used in engineering, management, learning, and problem-solving.

- Plan – Decide what you want to do and how.
- Do – Execute the plan (try it out).
- Check – Review the results (did it work as expected?).
- Adjust (or Act) – Make changes to improve, then repeat.

Example: Article Summarization

- Plan: Outline key points you want in the summary.
- Do: Fetch and summarize the article.
- Check: Is the summary concise and accurate?
- Adjust: Shorten text, add references.
→ Repeat loop until the summary is sharp.



Behavior 1 — Reason + Act (ReAct)

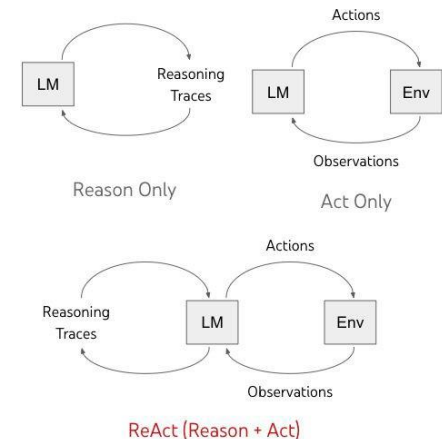
LLMs **think** (reason) and **use tools** (act) in steps.

Think of why GPT suddenly decides to browse the web even if you don't have an explicitly you need to do that. 'What is Apple's stock price today?'

Why it works: reasoning guides which tool to call; tools bring real data back.

How LLMs Think & Act

- **Reasoning (Think):** LLMs break down a problem step by step.
- **Acting (Do):** They decide which **tool** to use (e.g., web search, database, calculator, API).
- **Loop:** Reasoning guides which tool to call → Tool brings back real data → LLM uses it to refine the answer.



LangChain:

<https://python.langchain.com/docs/introduction/>

LangChain is a framework for developing applications powered by large language models (LLMs). Think of LangChain as the application-building layer around the language model.

LangChain simplifies every stage of the LLM application lifecycle:

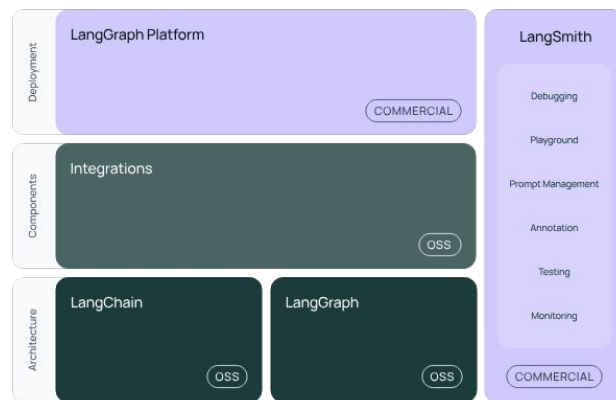
- Development: Build your applications using LangChain's open-source **components** and **third-party integrations**. Use **LangGraph** to build stateful agents with first-class streaming and human-in-the-loop support.
- Productionization: Use **LangSmith** to inspect, monitor and evaluate your applications, so that you can continuously optimize and deploy with confidence.
- Deployment: Turn your LangGraph applications into production-ready APIs and Assistants with **LangGraph Platform**.

LangChain: build.

LangGraph: orchestrate.

LangSmith: observe and evaluate.

Deployment platform: run it in production.

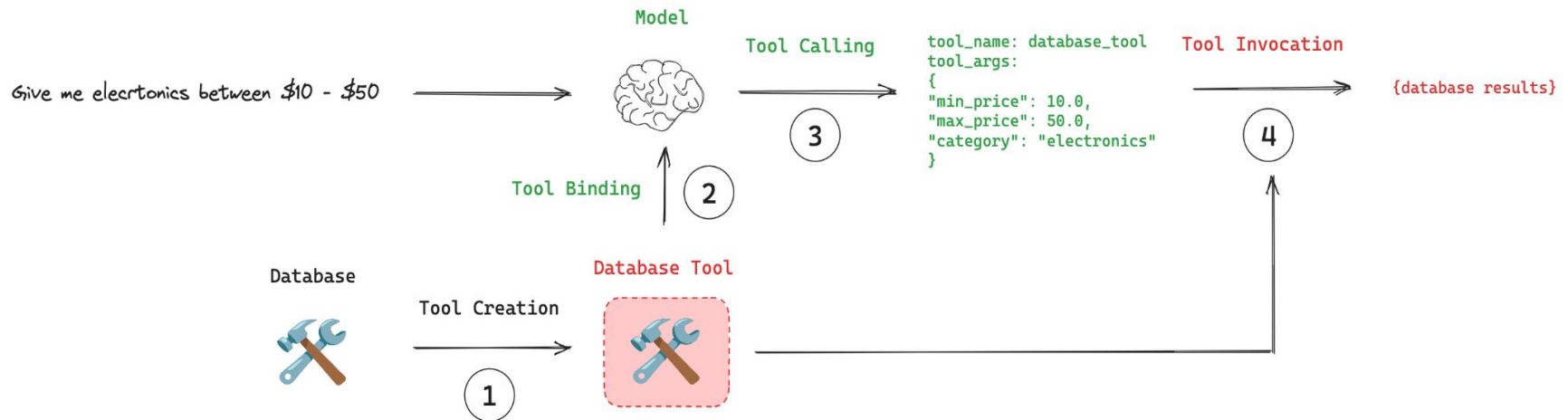


Tool Calling in LangChain

https://python.langchain.com/docs/concepts/tool_calling/

Key concepts

1. Tool Creation: Use the `@tool` decorator to create a `tool`. A tool is an association between a function and its schema. `@tool def search_products(min_price: float, max_price: float, category: str):`
2. Tool Binding: The tool needs to be connected to a model that supports tool calling. This gives the model awareness of the tool and the associated input schema required by the tool. `llm_with_tools = llm.bind_tools([search_products])`
3. Tool Calling: When appropriate, the model can decide to call a tool and ensure its response conforms to the tool's input schema.
4. Tool Execution: The tool can be executed using the arguments provided by the model.



```
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0) #for deterministic output
llm_with_tools = llm.bind_tools([get_datetime])
```

Conversation where the model MUST use the tool **for** the date (**not** guess)

```
messages = [
    SystemMessage(content="Always call get_datetime for any date/time info."),
    HumanMessage(content=(
        "Create a short note that includes today's date in both Asia/Kolkata and
        America/Los_Angeles, " "then add one friendly sentence."
    )),
]
ai = llm_with_tools.invoke(messages)
tool_calls = ai.additional_kwargs.get("tool_calls", []) or []

# If the model only called one timezone, we can nudge it with another turn:
if len(tool_calls) < 2:
    # Ask it explicitly to also fetch the second timezone (teaches multi-call)
    messages += [ai, HumanMessage(content="Also get the time in America/Los_Angeles.")]
    ai = llm_with_tools.invoke(messages)
    tool_calls = ai.additional_kwargs.get("tool_calls", []) or []
```

```
#Execution
tool_results = []
for call in tool_calls:
    args = json.loads(call["function"]["arguments"] or "{}")
    tool_results.append(ToolMessage(
        tool_call_id=call["id"],
        name=call["function"]["name"],
        content=json.dumps(get_datetime.invoke(args))
    ))

final = llm_with_tools.invoke(messages + [ai] + tool_results)
print(final.content)
```

Tool Calling: https://python.langchain.com/docs/concepts/tool_calling/

Output



Note:

- Today's date in Asia/Kolkata is August 29, 2025 (Friday) at 02:13.
- Today's date in America/Los_Angeles is August 28, 2025 (Thursday) at 13:43.

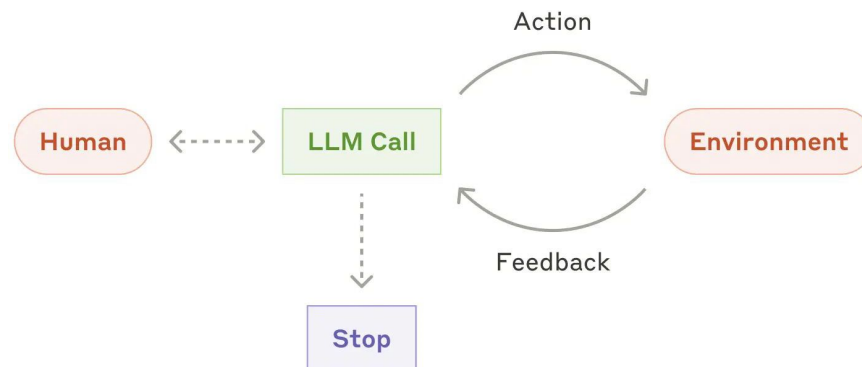
Have a wonderful day filled with joy and laughter!

Behavior 2 — Memory & Reflection

- Agents don't just act — they remember past experiences, evaluate what worked, and self-critique before giving the next reply.
- This creates a feedback loop:
Store → Evaluate → Improve → Repeat
- Memory + Reflection helps agents adapt across interactions → not just reactive, but adaptive and improving.

Example: Writing Assistant

- **Store:** Keeps track that the last draft was too long.
- **Evaluate:** “Feedback said it needed to be concise.”
- **Improve:** Produces a shorter, tighter draft next time.



Memory & Reflection — pick the right store (what to use, when)

Agents need memory to **recall**, **reflect**, and **improve**.

Different vector stores are suited for different use cases:

Semantic / RAG (similarity search): Core idea: Store text → retrieve by meaning, not keywords.

- FAISS (local, Python lib): best for small datasets, single machine, fast prototyping.
- Chroma (local/server): good for quick dev projects; minimal ops.
- PGVector (Postgres extension): Mix vectors + SQL filters/joins together; Great when metadata matters. Example: Search documents by both content + date tags.
- Qdrant / Milvus (self-hosted): Scalable, high-QPS vector search on your own infra.
- Weaviate / Pinecone (managed): production scale without ops; pay-as-you-go. Example: SaaS chatbot serving thousands of customers.

PGVector

metadata in **PGVector** usually refers to **extra fields stored alongside embeddings** that you can filter or join on using SQL. Examples of Metadata for PGVector: Document Metadata, title, author, source_url, Time Metadata created_at, updated_at, year

With PGVector you can do:

```
SELECT content
FROM documents
WHERE topic = 'AI' AND created_at > '2024-01-01'
ORDER BY embedding <-> query_embedding
LIMIT 5;
```

Query: "Explain REST APIs"

query_embedding → [0.07, -0.22, 0.91, ...]

Database finds chunks with embeddings close to it, e.g.: "REST allows communication via HTTP methods"

This combines **semantic similarity** with **structured filters** — super powerful for **metadata-heavy RAG**.

When to use a Vector DB

Semantic Recall

- You need to find text by **meaning**, not just exact keywords.
- *Example:* Query: “cheap flights” → Matches “low-cost airfare”.

Re-ranking / Hybrid Search

- Combine **keyword search (lexical)** + **semantic search (vectors)** for better accuracy.
- *Example:* Search for “Python data analysis” → Lexical hits + semantic matches from tutorials.

Metadata Filters

- Filter results with structured fields like author, date, tags.
- *Example:* “Find research papers about AI published after 2022.”

Graph / Entity memory (relationships)

Graph / Entity Memory (relationships)

- **Tools:** Neo4j, AWS Neptune, TigerGraph
- **RDF triple stores (GraphDB (ontotext)):** when ontologies & reasoning are needed.
- **Use When:**
 - You need to traverse multi-hop relationships: *“User → Project → Constraint”*
 - Answer questions like *“Who connects to what, and why?”*
- **Example:**
 - *Find all developers working on projects that depend on Database X.*

Full-text / Hybrid Search

- **Tools:** Elasticsearch, OpenSearch
- **What they offer:**
 - Keyword + BM25 ranking, Aggregations & filters (facets), Hybrid search (KNN + lexical)
- **Use When:**
 - You need log analytics, dashboards, or combining keyword + semantic search.

Key Idea:

- **Graph DBs** = best when **relationships matter**.
- **Full-text / Hybrid Search** = best when **keywords, filters, and aggregations** are

Behavior #3 — Delegation

Break work across **specialized agents** (Example: planner, researcher, coder).

A2A — what is Agent-to-Agent?

Agents message each other to solve tasks, sometimes with a supervisor or a group chat manager deciding who speaks next.

A2A = agents collaborating like a team.

They pass messages, share results, and build on each other's work.

MCP vs A2A — two protocols, two jobs

MCP (Model Context Protocol) — how an agent reaches its tools and data

- An open standard for exposing tools, resources and prompts to an LLM app.
- The agent asks a server: what can I call, and how? One agent, many capabilities.
- Your FastAPI service is exactly this surface — typed inputs, validated bodies,
 - an OpenAPI schema the model can read before it calls anything.

A2A (Agent-to-Agent) — how an agent reaches its peers

- A protocol for agents to advertise skills, delegate a task and return a result.
- The agent asks another agent: can you take this? Many agents, one goal.
- The ping-pong and supervisor patterns on the next slides are A2A.

They compose, they do not compete

- A single agent typically uses MCP downward to its tools and A2A sideways
 - to its peers — the supervisor delegates over A2A, each worker calls tools over MCP.
- Rule of thumb: MCP answers "what can I use?" — A2A answers "who can help me?"

A2A — ping-pong (to & fro context passing)

What Happens:

1. **Agent A** → sends a message with context.
2. **Agent B** → replies, adds reasoning or output.
3. The **chat history/summary** is shared back so both agents see the updated state.
4. Repeat this back-and-forth until a **stop rule** is met (e.g., task complete or max turns).

Example: Debugging a Function

- **Agent A (Coder):** “Here’s the function, but it throws an error.”
- **Agent B (Reviewer):** “The issue is a missing import. Add `import os`.”
- **Agent A:** Updates code, posts new version.
- **Agent B:** Runs tests again, confirms fix.
- Stop when Reviewer approves

Key Idea

Ping-pong = agents **pass context back and forth**, each improving or correcting, until the goal is reached.

Minimal Example

A2A Ping-Pong (Fibonacci Task) : Goal: *“Write Python to compute the 10th Fibonacci number.”*

Turn 1 — Analyst (A): “Plan: Use recursion or iteration. Keep it simple with a loop.”

→ Sends plan to Coder (B).

Turn 2 — Coder (B): “Wrote a loop version. Code runs but missing print statement.”

→ Returns code sketch + status.

Turn 3 — Analyst (A): “Context summary: last turn = loop works, missing print.

TODO: add print.” → Suggests fix.

Turn 4 — Coder (B): “Updated code. Now prints the 10th Fibonacci number = 55.”

→ Reports success.

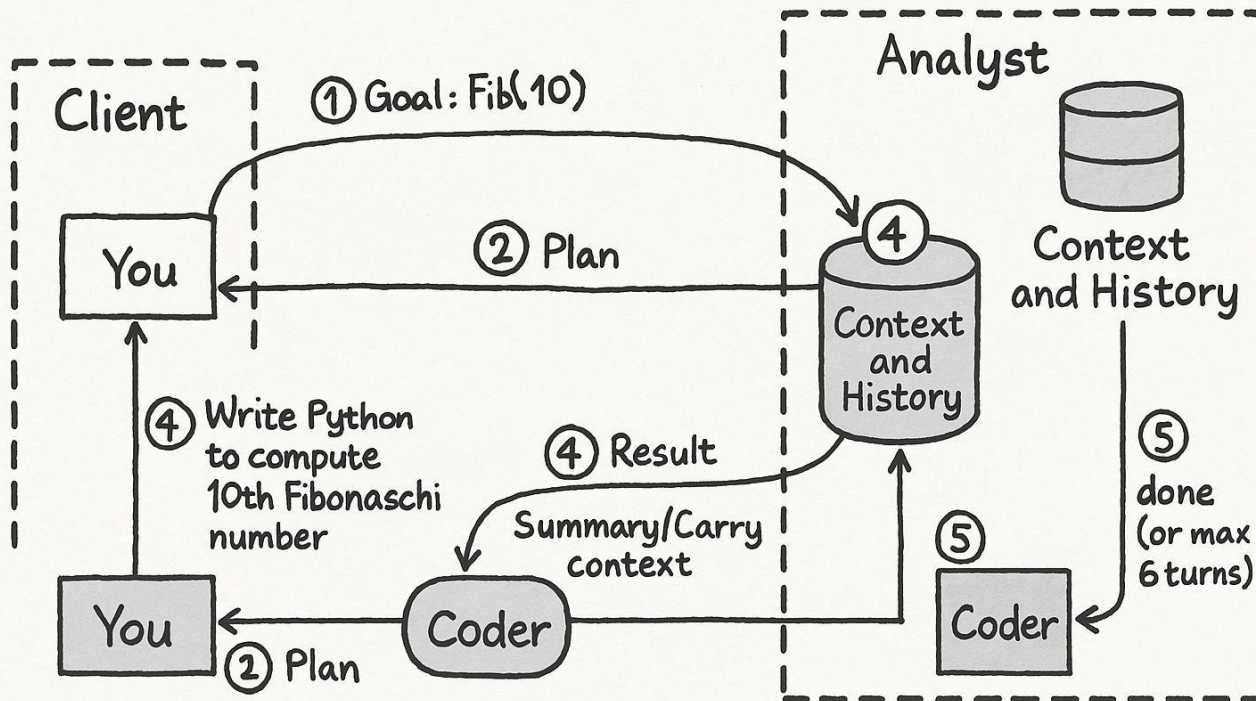
Carry Context Each Turn: Summarize what changed + open TODOs → keeps loop efficient.

Stop Rule:

- End when Coder outputs final working code
- Or max 6 turns to prevent infinite ping-pong.

Ping-Pong A2A = goal → back-and-forth refinement → stop when done.

A2A — minimal “ping-pong”



A2A with graphs — LangGraph supervisor pattern

- Agents are graph nodes (Planner, Researcher, Coder, Reviewer).
- A **Supervisor LLM** is a routing node: it reads current state and decides the next node.
- Edges encode possible next hops; the graph executes step-by-step until a stop rule.

What state does the next agent see?

You choose the state model. Two common patterns:

1) Pass messages (chat-style state) State: `messages[ ]` (running conversation) + optional `scratchpad`.

- Next agent sees: the latest summarized transcript (or last N turns) plus any role tags.
- Pros: Natural, debuggable, model-agnostic.
- Cons: Token growth; you must summarize/trim.

2) Pass tool args / structured fields (form-style state)

- State: typed fields the next agent needs (e.g., `papers[ ]`, `summaries[ ]`, `endpoint_spec`).
- Next agent sees: only the fields needed; not the whole chat.
- Pros: Compact, deterministic, great for long pipelines.
- Cons: Less conversational flexibility; you must design the schema.

```
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0) parser =  
StrOutputParser()  
  
# Prompts  
researcher_prompt =  
    ChatPromptTemplate.from_messages([  
        ("system",  
         "You are a precise researcher. Extract 3-5 concise facts relevant to the  
         user's goal. "  
         "Use short bullets only. No preamble, no summary."), ("human",  
         "Goal: {goal}\n\nReturn only bullet facts.")  
    ])  
  
writer_prompt = ChatPromptTemplate.from_messages([  
    ("system",  
     "You are a clear, concise technical writer. Using the provided facts, "  
     "write a 3-4 sentence summary for a technical audience. End with  
     one caution."),  
    ("human", "Goal: {goal}\n\nFacts:\n{facts}\n\nWrite the final  
summary.")  
    ])
```

Output

=== TRACE (who said what) ===

01 [22:04:37] [USER] (goal)

Goal: summarize agent-to-agent (A2A) interactions

02 [22:04:37] [SUPERVISOR] (route)

Routing to researcher (facts first).

03 [22:04:38] [RESEARCHER] (facts)

- A2A interactions involve direct communication between autonomous agents without human intervention.
- These interactions can enhance efficiency in tasks such as data sharing, decision-making, and resource allocation.
- A2A communication protocols often utilize standards like MQTT or CoAP for lightweight messaging.
- Security in A2A interactions is critical, often implemented through encryption and authentication mechanisms.
- A2A systems are commonly used in IoT applications, robotics, and distributed AI networks.

Output (Continued)

04 [22:04:38] [SUPERVISOR] (route)

Facts gathered; routing to writer.

05 [22:04:40] [WRITER] (final)

Agent-to-agent (A2A) interactions facilitate direct communication between autonomous agents, enhancing efficiency **in** tasks such as data sharing,

decision-making, and resource allocation.

These interactions typically employ lightweight messaging protocols like MQTT or CoAP,

while prioritizing security through encryption and authentication mechanisms.

A2A systems are prevalent **in** IoT applications, robotics, and distributed AI networks. Caution should be

exercised regarding potential vulnerabilities **in** security implementations, as they can expose systems to risks.

Guardrails for agent behavior & A2A chats

Add programmable guardrails between app code and the LLM.

Similar to a single agent situation: topic control, jailbreak detection, PII redaction.

Topic control / policy gates

- Allowlist: “Only answer about course topics: HTML, REST, FastAPI...”
- Denylist: Block “malware creation,” “grading policy leaks,” etc.

Jailbreak & prompt-injection detection

- Classifier/regex on inputs: flag “ignore previous instructions,” “system override,” etc.
- If flagged → route to safe reply or human review.

PII detection & redaction

- Detect emails, phone #s, SSNs; replace with [REDACTED].
- Example: Before an agent posts to group chat, scrub PII.

A2A specifics

- Supervisor uses guardrails to decide who speaks next (route only to allowed nodes).
- Ping-pong chats: run both directions through guardrails (A→B and B→A).
- Shared state/blackboard: scrub PII on write; enforce TTL, size limits, and field schema.

Guardrails sits between your app and the LLM.



Application code interacting with LLMs through programmable guardrails.

```
DENY = {"passwords", "credit card"}
def allow(user_msg: str) -> bool:
    return not any(bad in user_msg.lower() for bad in DENY)

msg = "Can you store my credit card?"
print("Blocked" if not allow(msg) else "Allowed")
```

Evaluate A2A behavior

1) Traces (who spoke, when, why)

- Log per hop: {ts, speaker, input_hash, output_hash, tools_used, reason}
- Reason strings: short “why I acted” notes (route decisions, tool choices).
- Example row:
12:01:04 | Supervisor -> Coder | reason="needs code for /summarize"

2) Success criteria

- Task finished? Stop rule met (status=done, artifact produced, tests pass)
- Tool errors? Rate, types, retries, backoffs
- Loops avoided? Max turns not exceeded; no oscillation ($A \leftrightarrow B \rightarrow A \leftrightarrow B$ with no state change)
- Latency & cost: per-agent, per-tool, per-episode budgets

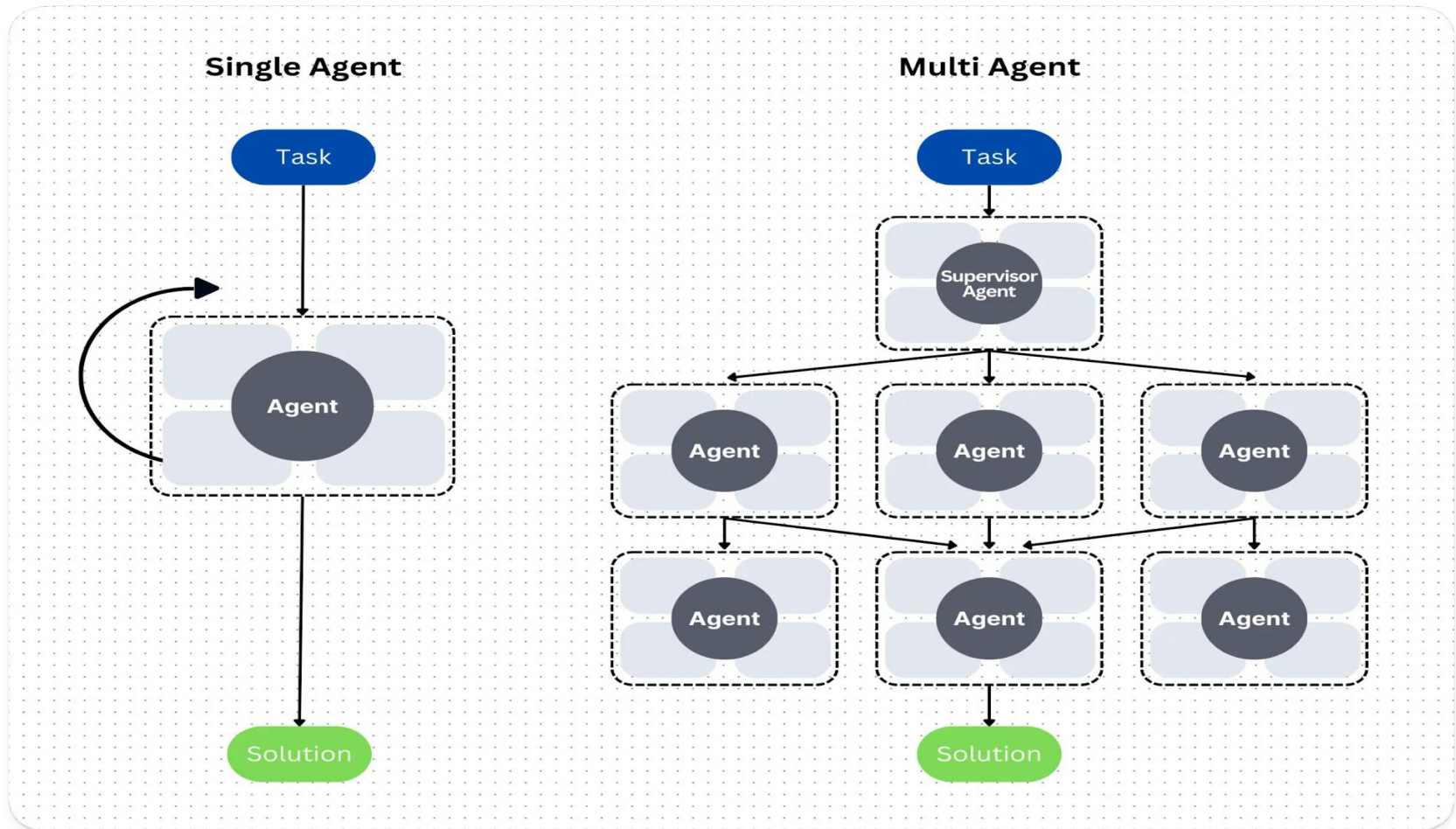
3) Unit tests for agents (role fixtures, a fixture = predefined setup or data used for repeatable tests)

- **check: pass fixtures through each role?**
- Planner: given a goal fixture, returns ordered steps with acceptance criteria
- Researcher: given a query fixture, returns sources + citations (schema-validated)
- Coder: given a spec fixture, returns runnable code + tests
- Reviewer: given outputs, returns pass/fail + actionable diffs
- Tip: run these headless (no other agents), then in integration with the graph.

4) Spot-check ReAct (reasoning $\leftrightarrow$ action alignment)

- **Check: did reasoning justify the action?**
- Red flags: “I’ll browse” with no freshness need; calling Python for trivial math; hallucinated sources.
- Pass example:
Reason: “Stock price requested for today $\rightarrow$ knowledge may be stale $\rightarrow$ browse web.”
Action: `web.search(ticker)`; Evidence logged; Answer cites fetched data.

the multi-agent system preferred over the single-agent system



One agent, powered by an LLM, analyzes survey responses for sentiment and nuances, detecting elements like frustration or satisfaction.

Another agent, specialized in crafting dynamic queries, generates follow-up questions aligned with the detected sentiment, user profile, and product feature.

For instance, if frustration about a product feature is identified, the follow-up might delve deeper into the specific pain point that applies to the user profile.